

Practical 11

Race-condition version

```
gcc -Wall -Wextra -pthread race_counter.c -o race_counter
./race_counter
```

The unsynchronized shared increment has a data race. The actual value may be lower than the expected value.

Mutex version

```
gcc -Wall -Wextra -pthread mutex_counter.c -o mutex_counter
./mutex_counter
```

The mutex serializes the critical section, so the final count should equal the expected value.

Run several times to demonstrate the difference.

Mutex_counter.c-

```
#include <pthread.h> #include <stdio.h> #include <stdlib.h>

#define THREADS 4 #define ITERATIONS 1000000

static long counter = 0; static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *arg) { (void)arg; for (long i = 0; i < ITERATIONS; i++)
{ pthread_mutex_lock(&lock); counter++; pthread_mutex_unlock(&lock); } return
NULL; }

int main(void) { pthread_t t[THREADS];

for (int i = 0; i < THREADS; i++)
    if (pthread_create(&t[i], NULL, worker, NULL) != 0) {
        perror("pthread_create"); return 1;
    }

for (int i = 0; i < THREADS; i++)
    pthread_join(t[i], NULL);

printf("Expected: %ld\n", (long)THREADS * ITERATIONS);
printf("Actual:   %ld\n", counter);
```

```
pthread_mutex_destroy(&lock);  
return 0;
```

```
}
```

Race_counter.c-

```
#include <pthread.h> #include <stdio.h> #include <stdlib.h>
```

```
#define THREADS 4 #define ITERATIONS 1000000
```

```
static long counter = 0;
```

```
static void *worker(void *arg) { (void)arg; for (long i = 0; i < ITERATIONS; i++) counter++;  
return NULL; }
```

```
int main(void) { pthread_t t[THREADS];
```

```
for (int i = 0; i < THREADS; i++)  
    if (pthread_create(&t[i], NULL, worker, NULL) != 0) {  
        perror("pthread_create"); return 1;  
    }
```

```
for (int i = 0; i < THREADS; i++)  
    pthread_join(t[i], NULL);
```

```
printf("Expected: %ld\n", (long)THREADS * ITERATIONS);  
printf("Actual:    %ld\n", counter);  
return 0;
```

```
}
```